

P395: Guide 5: Root finding / Extremization

1. Start up a new notebook and title it 'Guide5_your_name'. Enter

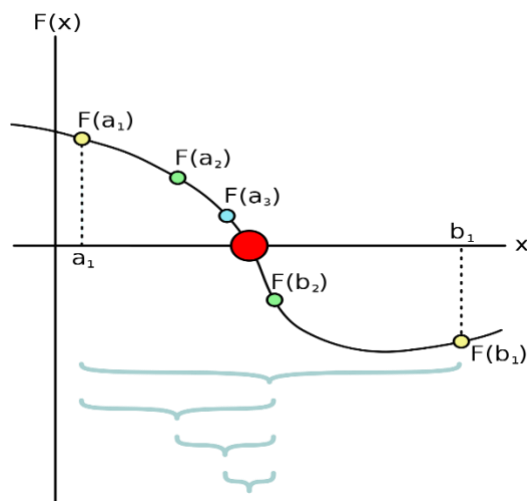
```
import matplotlib.pyplot as plt  
  
import numpy as np
```

Background Info: Root finding

There are many problems in physics where for a system we want to find a set of parameters that minimize or maximize some property. There are also problems where we have to solve a nonlinear equation, and an easy closed-form expression for the solution is not possible. Such problems are related to finding the roots of a function. Here we will look at several numerical methods that are suited for finding roots of functions and can also be applied to minimizing or maximizing a scalar function.

For the case of finding the roots of a function $f(x)$, we are interested in finding values of x that satisfy $f(x) = 0$. (Here we are just considering scalar functions of one variable, but below we will look at multidimensional examples; the methods for 1D systems generalize in a straightforward way to multidimensional functions and systems of equations.) We start by assuming there is just a single root in an interval $x \in [a, b]$ (in general, finding all the roots of a function is non-trivial). All root-finding methods are built on iterative procedures: one starts with a guess x_0 for a root and uses some update rule to come up with a new guess. This is iterated until convergence (*i.e.*, your updated value for x doesn't change much).

Task 1: Bisection Method



The simplest and most robust root-finding method is the *bisection method*. If there is a root between $[a, b]$, this method will find it, although not necessarily very rapidly. It also relies on the function changing sign over the interval, namely $f(a)f(b) < 0$, as shown in the figure. (This need not always be the case, in which case the bisection method wouldn't work. *E.g.*, $(x - 1)^2$ has a root $x=1$ but is never negative.) The idea is to calculate the midpoint c (bisection), which gives two intervals $[a, c]$ and $[c, b]$, then pick the interval on which the product of the function at the two points is negative. This new interval is then bisected, and the procedure is iterated until the distance between the interval endpoints is less than some tolerance. This is shown in the figure.

1. Consider the function $\sin(\cosh(2x))$. Plot this function on the interval $[0, 1]$.

2. Analysis: Does it have a root? If so, how many?
3. Write code that implements the bisection method (described above).
4. Use your bisection method code to find the root of $\sin(\cosh(2x))$ on $[0,1]$. How many iterations did it take?

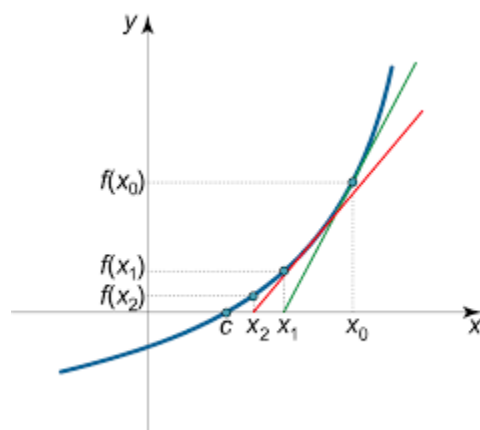
Python's `scipy.optimize` has a number of routines for root finding and minimization, including the bisection method.

5. Import `scipy.optimize` and look up how to use the function `bisect`.
6. Use `bisect` from `scipy.optimize` to find the root of $\sin(\cosh(2x))$ on $[0, 1]$. Set `full_output=True` to see how many iterations it took.

Task 2: Newton-Raphson

The bisection method is a robust method for finding a root on an interval, even for discontinuous functions; however, it can be a bit slow, especially if it needs to be called repeatedly for a particular problem. A method that converges faster but requires the use of the derivative of the function and a good initial guess of the root is the Newton-Raphson (NR) method. It is based on a Taylor expansion. Say we start with a guess x_i which is close to the actual root at $x_i + \epsilon$. Taylor expanding,

$$0 = f(x_i + \epsilon) = f(x_i) + \epsilon f'(x_i) + \dots$$



Stopping the expansion at first order, this equation says take the value of your function at your guess and then take a step, in the direction of the slope of f at your guess, that takes you to $y=0$. Where you wind up on the x -axis is your new guess. This is shown in the figure. So, an estimate for the step we should take towards the root is given by

$$\epsilon = -\frac{f(x_i)}{f'(x_i)}$$

Our update rule for the root is

$$x_{i+1} = x_i - \frac{f(x_i)}{f'(x_i)}$$

The drawback of NR is that if the initial guess is too far from the real root, this iterative procedure might not converge (or it may go to a different root).

Let's use NR to find the root of $\sin(\cosh(2x))$ on $[0,1]$. Instead of a range, you pass an initial guess to NR.

1. Analysis: Looking at the plot of $\sin(\cosh(2x))$, is there a range of x -values that would be bad initial guesses on $[0,1]$?
2. For the NR method, you also need a function that returns the derivative. What is the derivative of $\sin(\cosh(2x))$? Write a function that returns the derivative at x .
3. Look up how to use `newton` from `scipy.optimize`, where the function that calculates the derivative is also passed as an argument. Use the Newton-Raphson method to find the root of

$\sin(\cosh(2x))$ on the interval $[0,1]$. Use `full_output = True` to see how many iterations it takes.

Task 3: Other root-finding methods

Secant Method

This uses the same update rule as Newton-Raphson but uses a linear approximation of the derivative based on the values of the function. The secant method works in the absence of a closed-form expression of the derivative. Calling `newton` without a derivative function automatically does the secant method.

1. Now call `newton` on the above problem without passing a function that calculates the derivative, but using your same initial guess. This will run the secant method. Use `full_output = True` to see how many iterations it took.

Brent's Method

A robust and quick root-finding method is Brent's method. It uses a combination of the bisection and secant methods to find the root. In `scipy.optimize` this is the `brentq` function.

2. Look up the documentation on `brentq` and use it to find the root of $\sin(\cosh(2x))$ on $[0,1]$. Use `full_output = True` again to see how many iterations it took. It should be markedly faster than bisection.

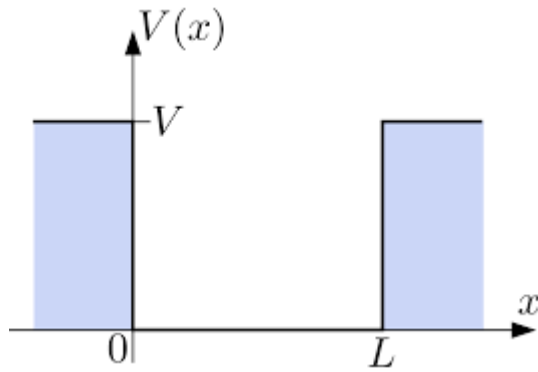
General-purpose root-finding routine

For nonlinear functions of more than one variable or possibly systems of nonlinear equations, there is a general-purpose root-finding routine called `root`. It uses methods that are generalizations of Brent's method (or NR) in higher dimensions. Happily, it also can compute roots of scalar functions of one variable too.

3. Look up the documentation on `root` and use it to find the root of your function above using the same initial guess that you gave to NR and secant.

Task 4: Application of root finding: Particle in a finite rectangular well

Here you will use one of the above root-finding algorithms to solve a physics problem that requires finding the roots of a nonlinear function.



Consider a quantum particle in a finite-depth (rectangular) potential well. Classically, if the particle has an energy less than the height of the potential well V , it is stuck in the box and can never get out; however, a quantum particle in such a potential has a finite probability of being outside the box! Not only that, but unlike a classical particle that could have a continuous range of energies (*i.e.*, its kinetic energy) in the box, the quantum particle does not: it can only have certain energies when it is in the box.

From Schrödinger's equation, the allowed energies E for a quantum particle of mass m trapped in a finite potential well ($0 < E < V$) of width L are solutions of the nonlinear equations

$$\tan \sqrt{L^2 m E / 2 \hbar^2} = \sqrt{(V - E) / E} \quad (\text{for even-numbered states})$$

or

$$\tan \sqrt{L^2 m E / 2 \hbar^2} = -\sqrt{E / (V - E)} \quad (\text{for odd-numbered states})$$

Consider an electron ($m = 9.1094 \times 10^{-31}$ kg) in a well of $V = 5$ eV and $L = 1$ nm. We'll use graphical methods to estimate the solutions first.

1. Plot the function on the left side of the above equations on $0 < E < 5$ eV (since $\tan x$ has discontinuities on this range, I would make it a scatter plot and set your y-axis limits to $(-6, 6)$). Then, on the same graph, plot the functions on the right sides of the above two equations (one for the even states and one for the odd states) versus E .
(Note: instead of looking up the physical constants and typing them in, `scipy.constants` has a number of them, including the mass of the electron and $\hbar$. You could import them and use the constants instead of typing them in yourself. Also, watch your units for consistency. If you use SI units for all the physical units, you'll need to convert your energy in eV to Joules for the left-side function. The right-side functions are a ratio of energies, so you just need to make sure E and V are in the same units.)
2. Analysis: The curves intersect at the allowed energies E (*i.e.*, the roots). How many allowed energies are there? From the graph, make rough estimates for their values.
3. Using each of your graphical estimates as initial guesses, use `newton` (*i.e.*, the secant method, without explicit use of the derivative) to find the above allowed energies, in units of eV.

Extremization = Minimization/Maximization

Instead of finding roots of a function, we might want to minimize it (*e.g.*, energy) or maximize it (*e.g.*, entropy) as a function of the system's parameters. Extremization of a function borrows all the same ideas and methods from root finding. For extremization we want to find locations where the derivative of the function is zero, $f'(x) = 0$, which is a root-finding problem. Applying Newton-Raphson to this, the update rule would be

$$x_{i+1} = x_i - \frac{f'(x_i)}{f''(x_i)}$$

So this requires knowledge of the 2nd derivative of the function as well. Iterating this equation from some starting value will lead to some extremum of the function. You then must look at the 2nd derivative to determine if it is a local minimum or maximum. If your function has many extrema, there are no guaranteed methods for finding the global extreme value; the best one can do is find a local extremum. Methods such as simulated annealing and other basin-hopping methods try to help move from one minimum to a better one (hopefully the global optimum).

Task 5: Minimization of functions

In `scipy.optimize`, there are several routines for finding the minimum of a function. (Note: if you want to find the maximum of a function $G(x)$, that is the same as finding the minimum of $-G(x)$.) For minimization of scalar functions of one variable, the methods apply the secant method and bisection to converge on the minimum. For extremization, there is also a method that is equivalent to bisection called 'golden search'. Briefly, if there is a minimum in the region $[a,c]$ then there is a point b such that $a < b < c$ and $f(b) < f(a), f(c)$. Given such a triplet of points (our initial bracketing), (a,b,c) , pick a point x in $[b,c]$. If $f(b) < f(x)$, then a minimum lies in $[a,x]$, so update the bracketing to (a,b,x) . But if $f(x) < f(b)$, then a minimum lies in $[b,c]$, so update the bracketing to (b,x,c) . Keep repeating this process until one converges on the minimum. It's called 'golden' because one can work out that the best choice for the location of x is such that $(c-x)/(x-b)$ is the golden ratio.

1. Analysis: Plot the function $f(x) = 2.1x^2 - 3.4x + 2.6$ so you can see where its minimum is located. Then, find its minimum using calculus.
2. Look up how to use `scipy.optimize.minimize_scalar`. Find the minimum of $f(x)$ using two methods, once using `method='brent'` and the other using `method='golden'`.
3. What do you notice about the number of iterations between the two methods? Did the answers agree with what you found in 5.1?

A more general method that can find the minimum of a function of many variables is `scipy.optimize.minimize`. It uses techniques borrowed from Newton-Raphson, namely using the direction of the derivative of the function to choose the direction of the steps. It works perfectly well on functions of a single variable.

4. Look up how to use `scipy.optimize.minimize`. Use it with the default settings to find the minimum of the above function. Instead of a bracket you must pass some initial guess for where the minimum might be located.

Applications of Extremization

Now we'll use the minimization routines to look at some physics problems where finding the minimum or maximum of a function is required.

Task 6: Landau free energy of phase transitions (revised from a problem by Malcolm Kennett)

Many systems in nature are seen to undergo phase transitions as some parameter(s) are changed. This is characterized by a change in some macroscopic physical property of the system known as the *order parameter*. For a ferromagnet, this corresponds to the bulk magnetization, and it spontaneously changes from zero magnetization at high temperatures to a non-zero magnetization at all temperatures below a critical temperature T_c . This is a 2nd-order phase transition, in that the order parameter changes continuously but its derivative is discontinuous at the transition.

Landau came up with a mean-field model to explain the phenomenon. The Landau free energy for a ferromagnet in terms of the bulk magnetization (order parameter) M is

$$F(M) = aM^2 + bM^4 - HM$$

where $a = a_0(T - T_c)$ is linearly dependent on the critical temperature T_c and flips sign as one crosses over from above to below T_c . You'll see that this flipping in sign is important. H is an external magnetic field, and b is a positive constant, independent of temperature.

1. Analysis: Rescale the free energy, so that the scaled free energy, $\tilde{f}(M) = F(M)/(a_0T_c)$, is in terms of $t = T/T_c$, $\tilde{b} = b/(a_0T_c)$ and $\tilde{h} = H/(a_0T_c)$.
2. Plot $\tilde{f}(M)$ versus M at $t = 0.4, 0.8, 1.0, 1.4$ using $\tilde{b}=0.07$ and $\tilde{h}=0$.
3. Analysis: What happens to the shape of $\tilde{f}$ as you go from below T_c to above T_c ?
4. At a given temperature, the equilibrium magnetization occurs when $\tilde{f}$ is a minimum. Plot the equilibrium magnetization as a function of t using $\tilde{b}=0.07$ and $\tilde{h}=0$. (HINT: To do this, use a range of t , and for each t use `minimize` to find the value of M where $\tilde{f}(M)$ is minimum using an appropriate choice for the initial M_0 .)
5. Analysis: From your plot in 6.4, does M satisfy the conditions of a 2nd-order phase transition from this model?

Below the critical temperature this system can be bi-stable (*i.e.*, it has 2 minima) at certain values of the magnetic field h . Bi-stable systems display hysteresis (*i.e.*, memory) as one of the system parameters is changed. For a ferromagnet this corresponds to the magnetic field h .

6. With $t=0.6$ and $\tilde{b}=0.07$, plot $\tilde{f}(M)$ at $\tilde{h} = -1.0, 0.0$, and 1.0 . What happens to the shape of $\tilde{f}$?
7. Analysis: What do you think will happen to the equilibrium magnetization (*i.e.*, the minimum in $\tilde{f}$) as you drive the system from $\tilde{h} = -1$ to 1 and then back again? (HINT: you fall into the closest minima from where you start, *i.e.*, the minimum from the previous value of $\tilde{h}$.) Put your answer in a text cell.
8. Calculate the equilibrium magnetization as the magnetic field $\tilde{h}$ is cycled from negative values to positive values and then back to negative values. For the forward sweep, have $\tilde{h}$ go from -1.0 to 1.0 . At the initial $\tilde{h}=-1.0$, use a starting guess value of the magnetization of $M_0 = -1.0$. Use `minimize` to find the minimum. At each new value of $\tilde{h}$, use the previous value for M at the minimum as the new initial guess. Plot your equilibrium M 's vs. $\tilde{h}$'s for the forward sweep. Now go back the other direction, letting your $\tilde{h}$'s go from 1.0 to -1.0 , again using your last value of M at the minimum as your initial guess. Plot your equilibrium M 's vs. $\tilde{h}$'s for the backward sweep on the same graph (using a different color).
9. Analysis: Describe what you see in your plot? Why does this magnetic system have 'memory'?

Task 7: Ode to incandescent light: Optimal operating temperature for a lightbulb
(A variation on a problem by Mark Newman)

For many years, electric lightbulbs were built on the observation that all objects at thermal equilibrium at temperature T emit light. How much visible light gets radiated depends on the object's temperature. Max Planck came up with a formula to explain the power of light radiated at a certain wavelength λ by an object at temperature T . His formula is *Planck's law*,

$$I(\lambda, T) = 2\pi A h c^2 \frac{\lambda^{-5}}{e^{hc/(\lambda k_B T)} - 1}$$

where A is the area of the object, h is Planck's constant, c is the speed of light, and k_B is Boltzmann's constant. Visible light ranges in wavelength from $\lambda_1 = 390$ nm (violet) to $\lambda_2 = 750$ nm (red). The total power radiated in the visible spectrum at temperature T is $\int_{\lambda_1}^{\lambda_2} I(\lambda, T) d\lambda$, and the total power radiated is $\int_0^\infty I(\lambda, T) d\lambda$. Dividing one by the other gives the efficiency of visible light emitted. Doing a variable transformation and using the exact value for the integral of the total radiated power, one can work out the following integral for the efficiency at temperature T ,

$$\eta(T) = \frac{15}{\pi^4} \int_{hc/(\lambda_2 k_B T)}^{hc/(\lambda_1 k_B T)} \frac{x^3}{e^x - 1} dx$$

1. Write a function that calculates $\eta(T)$ at a given temperature T . You should use a numerical integration method such as `quad` to do the integral.
2. Plot $\eta(T)$ between $T=300$ K and $T=10,000$ K on a log scale. You should see that there is a temperature where the efficiency is maximal.
3. Analysis: From your plot, estimate what this temperature is and put your answer in a text cell.
4. Use `minimize` to find the T at which the efficiency is maximum. What is the efficiency at this temperature?
5. Incandescent lightbulbs use tungsten (the metal with the highest melting temperature) as the filament. Look up the melting temperature of tungsten.
6. What challenge does the optimal temperature pose for the operation of incandescent lightbulbs? From your plot, what is the maximum efficiency we might expect to get from a metal like tungsten?

Task 8: Wrap up

1. Please provide any feedback on questions that you found difficult to understand because of the wording.
2. Please provide any other comments about this notebook.